

JDBC Theory Questions:

- 1 What is RowSet?
- 2 Difference between JdbcRowSet and CachedRowSet?
- 3 What is connection pooling advantages?
- 4 What is fetch size?
- 5 What is pagination?
- 6 How to improve performance?
- 7 How to handle large data?
- 8 How to use stored procedures?
- 9 How to use batch updates?

- 10 How to reuse connections?

MCQ Questions

1.

```
Connection con = DriverManager.getConnection(url, user, pass);  
System.out.println(con != null);
```

- A. true
- B. false
- C. Error
- D. null

2.

```
Statement st = con.createStatement();  
int x = st.executeUpdate("INSERT INTO emp VALUES(1, 'A')");  
System.out.println(x);
```

- A. 0
- B. 1
- C. true
- D. Error

3.

```
ResultSet rs = st.executeQuery("SELECT * FROM emp");
while(rs.next()){
    System.out.print(rs.getInt(1));
}
```

- A. Error
- B. Prints first column values
- C. Prints all columns
- D. Nothing

4.

```
PreparedStatement ps = con.prepareStatement("INSERT INTO emp VALUES (?, ?)");
ps.setInt(1,1);
ps.setString(2,"A");
ps.executeUpdate();
```

- A. Error
- B. Inserts record
- C. Prints data
- D. Nothing

5.

```
System.out.println(rs.next());
```

- A. true/false
- B. row data
- C. error
- D. null

6.

```
con.setAutoCommit(false);
```

- A. Enables auto commit
- B. Disables auto commit
- C. Error
- D. Commit happens automatically

7.

```
con.rollback();
```

- A. Saves data
- B. Undo changes
- C. Deletes table
- D. Error

8.

```
con.commit();
```

- A. Undo
- B. Save changes
- C. Error
- D. Delete

9.

```
ps.executeQuery();
```

- A. Used for SELECT
- B. Used for INSERT
- C. Error always
- D. Delete

10.

```
con.close();
```

- A. Opens connection
- B. Closes connection
- C. Deletes DB
- D. Error

Practice Programs

1) Call a stored procedure from Java using JDBC.

Task:

- Create a stored procedure in DB (e.g., get student by ID)
- Call it using `CallableStatement`

Output:

Display returned record

2) Perform multiple SQL operations in a single batch.

Task:

- Insert/update multiple records
- Execute using `executeBatch()`

Output:

All queries executed successfully

3) Display records page-wise from database.

Task:

- Fetch limited rows using `LIMIT`
- Show 5 records per page

Example:

Page 1 → records 1–5

Page 2 → records 6–10

4) Efficiently retrieve large datasets from database.

Task:

- Use `setFetchSize()`
- Avoid loading entire data into memory

5) Reuse database connections instead of creating new ones.

Task:

- Implement basic connection pool logic
- Reuse connection for multiple queries

6) Improve performance of database operations.

Task:

- Use batch processing
- Use PreparedStatement
- Optimize queries

7) Transfer money between two accounts.

Task:

- Deduct from sender
- Add to receiver
- Use commit/rollback

Condition:

If any error → rollback

8) Display large data in pages.

Task:

- Use LIMIT and OFFSET
- Show user-friendly navigation

9) Fetch DB records and store them in CSV format.

10) Read CSV file and insert data into DB.